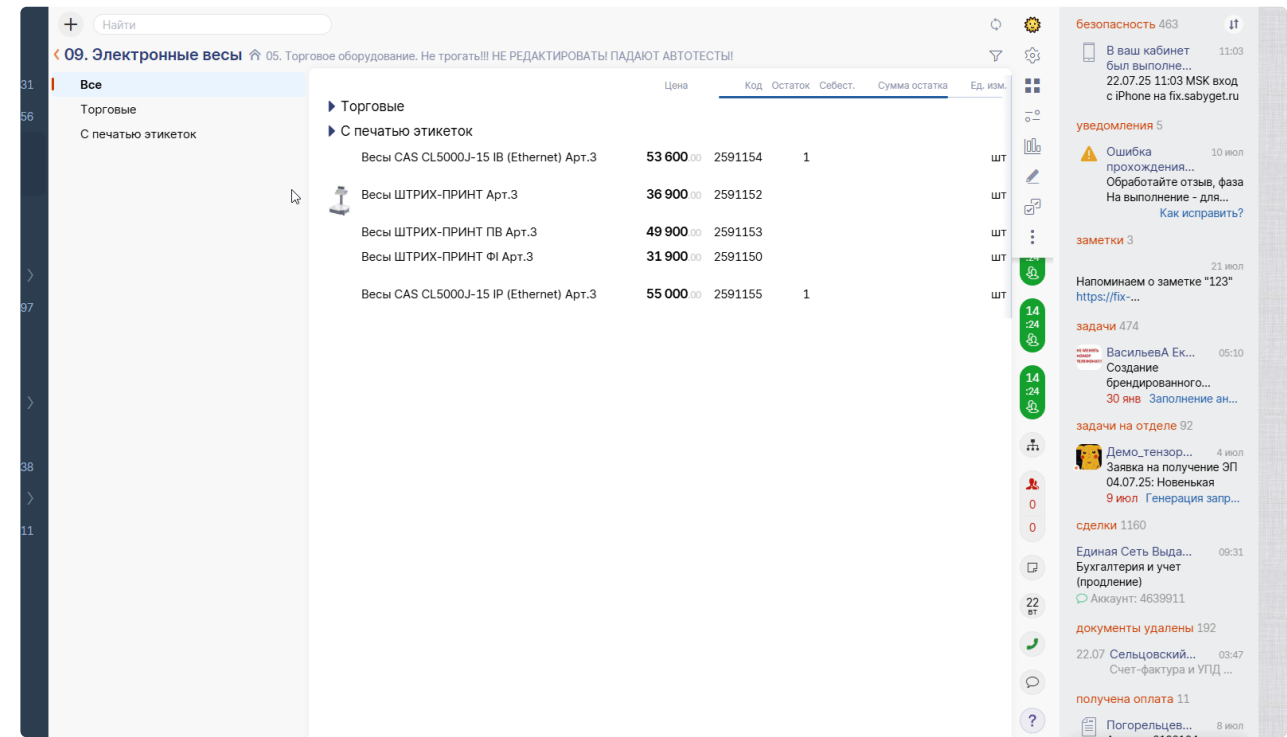


Механики работы с буфером обмена

Описание

Буфер обмена выполняет роль **своеобразного посредника**, позволяя пользователям выполнять команды "Копировать" и "Вставить" в рамках бизнес-процессов. Например, перенос клиентов между реестрами, заполнение товаров по корзине и др.



Концептуальное описание продукта Saby-буфер

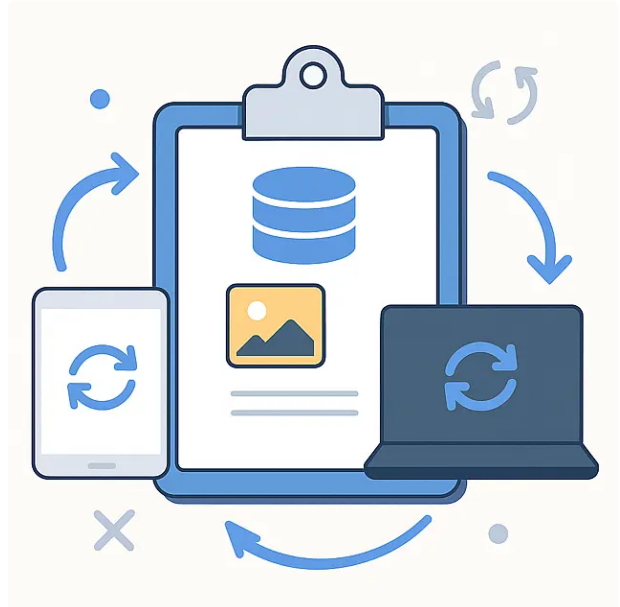
<https://link.sbis.ru/article/1e54e81c-c29e-4b29-a6fd-0dae29f69100/c114ad78-7590-460d-9793-d6169bf9851f>

Saby-буфер сохраняет скопированные данные в виде json-файлов (для списков и табличных данных) или графических файлов (для изображений в форматах PNG, JPEG и др.) на **СБИС Диске**.

Для каждого такого файла в буфере обмена формируется объект, содержащий тип данных и описание, необходимое для идентификации и обработки соответствующего содержимого.

▼ Пример

Содержимое объекта		Содержимое файла	
1	{	1	{
2	objectType: 'Contragent'	2	"d":
3	fileName:	3	["CRMClients.ListClientsOnline",
4	'Клиенты_CRM.json',	4	...]
5	attachId: '7d0ef674-e665-	5	"s":
6	475f-bba8-e090774a3714'	6	[{"t": "Строка", "n": "MethodName"},
	...		{"t": "Запись", "n": "Filter"},
	}		}

	<pre>6 { "t": "Запись", "n": "Navigation" }, 7 { "t": "Запись", "n": "Sorting" }, 8 { "t": "Запись", "n": "ExtFields" }], "_type": "record" 9 }</pre>
<pre>1 { 2 objectType: 'image', 3 fileName: "Скриншот1.png", 4 attachId: "541bd34c-4dc0-4cd7-accd-6cd27b322266", 5 ... 6 }</pre>	

В случае, если в буфер копируется **ссылка**, файл не создаётся — адрес ссылки сохраняется в свойствах объекта буфера как параметр.

Операции копирования и вставки

Поддерживается вставка из внешних источников (например, из Excel или текстового редактора).

Saby-буфер автоматически на основе параметра **objectTypes** определяет наличие в системном буфере обмена данных электронной таблицы (Excel / LibreOffice) или диапазона ячеек, и формирует из этих данных JSON. При отсутствии данных в системном буфере обмена будет обращаться к **DOCVIEW3/sbisBuffer**

В контексте приложений Saby, библиотека **SabyBuffer** предоставляет единый программный интерфейс для работы с буфером обмена, поддерживая команды **копирования** и **вставки**:

Команда	Описание действия	Базовый принцип работы	Горячие клавиши	Реализация
Копировать	Сохраняет выбранные элементы (например, товары) во внутренний буфер обмена.	Записывает данные в файл формата json и сохраняет его на Saby Диск	Ctrl+C	Платформенная команда SabyBuffer/actions:copy Подробнее
Вставить	Вставляет содержимое буфера обмена в текущий контекст (документ, список клиентов).	Извлекает данные об объекте из буфера обмена СБИС и на их основе формирует параметры для вызова метода бизнес-логики importFromBuffer . В случае использования системного буфера обмена, формирует JSON из данных	Ctrl+V	Платформенная команда SabyBuffer/actions:paste Подробнее

	скопированной извне электронной таблицы (Excel / LibreOffice) или диапазона ячеек	
--	---	--

Команды работают с учетом выбранного **режима работы буфера**, который определяет, в каком виде хранятся данные, и как происходит их обработка.

Режимы работы буфера обмена

На текущий момент буфер обмена поддерживает два режима работы:

Режим работы с загрузкой данных по запросу

В данном режиме в буфере обмена хранятся параметры для последующей начитки данных, которая будет вызвана при выполнении команды вставки из буфера обмена.

Применяется в случаях, когда буфер обмена используется в интерактивном режиме (вставку из буфера инициирует сам пользователь), а начитка данных не является трудозатратной. При этом копируемые данные в сыром формате занимают значительный объем памяти. Значит, целесообразнее хранить параметры выборки, а не данные в сыром виде.



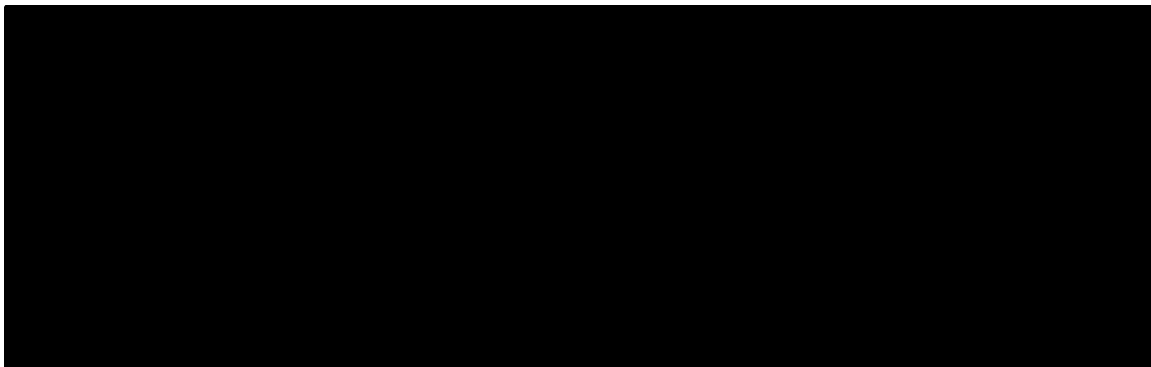
▼ Пример

В реестре [Клиенты](#) требуется поддержка работы с буфером обмена. Кейс подразумевает выделение нескольких клиентов и копирование их в буфер обмена.

В этом случае достаточно хранить параметры запроса, так как начитанные данные потребуются лишь в момент вызова пользователем команды вставки из буфера.

При этом процесс начитки списка клиентов не требует значительных ресурсов.

▼ Интерфейс, описывающий формат хранения параметров для начитки данных в буфере



Режим работы с сырыми данными

Данный режим требует бóльшего объема дискового пространства в сравнении с первым режимом, так как хранит непосредственно "сырые" данные.

Основное преимущество режима раскрывается при выполнении команды вставки из буфера - данные уже готовы к вставке и не требуют предварительной начитки.

Это позволяет значительно снизить нагрузку на серверную часть приложения в случаях, когда обращение к скопированным данным происходит при инициализации приложения (загрузке страницы).



▼ Пример

Корзина товаров позволяет пользователям собирать и хранить товары, которые они намерены приобрести. Когда пользователь выбирает товар, он может добавить его в корзину, что позволяет ему продолжать просматривать другие товары, не теряя при этом выбранные позиции.

Для хранения содержимого корзины используется буфер обмена.

В виду того, что корзина инициализируется при запуске приложения (открытия страницы), её содержимое (данные) необходимо сразу.

Таким образом, использование "сырых" данных позволяет обходиться без начитки и уменьшает нагрузку на Бизнес-логику каталога товаров.

▼ Интерфейс, описывающий формат хранения предварительно начитанных данных в буфере



Время жизни объекта в буфере обмена

В настоящий момент объект в буфере хранится 1440 минут (одни сутки). Определяется параметром облака *БуферСБИС.ВремяЖизниТранзакцииВБуфере*.

На данный момент нет возможности менять время жизни объекта в буфере обмена.

Команда "Копировать в буфер"

```
1 import { copy } from 'SabyBuffer/actions';
```

Команда "Копировать в буфер" позволяет пользователям поместить данные в специальное хранилище (так называемый СБИС-буфер).

▼ Режимы работы команды

На данный момент команда поддерживает два режима копирования:

1. Режим дополнения

Новый объект добавляется в буфер **перед уже существующими**, не удаляя их содержимое.

2. Режим замещения

Если в буфере уже есть объект **того же типа**, он удаляется, а вместо него помещается новый. Объекты других типов при этом не затрагиваются.



▼ Принцип хранения данных в буфере

Функция копирования записывает данные в файл формата **json** и сохраняет его на СБИС Диск.

В буфере хранится описание этого файла (ссылка на файл и тип сущности, информацию о которой файл содержит).

API

ICopyToBufferParams

Параметр	Тип	Описание
objectType	string	Тип сущности размещаемой в буфере. Используется для идентификации конкретного типа в общем буфере обмена. В буфере обмена могут лежать одновременно данные для корзины и для CRM. Их разделение друг от друга происходит за счёт значений данного свойства:

		- "Contragent" для CRM; - "Nomenclatures" для корзины.
dataType	TDataType (см. описание ниже)	Тип размещаемых данных
bufferObject	IBufferObjectQueryData IBufferObjectRawData	Данные, размещаемые в буфере обмена
fileName	string	Имя файла, в который записываются данные буфера обмена
copyMode?	TCopyMode (см. описание ниже)	Режим копирования - замещать объект в буфере обмена или добавлять новый объект перед предыдущим.



В буфере обмена могут располагаться одновременно несколько файлов с одинаковым именем.

TDataType

Значение	Описание
'query'	Режим работы с загрузкой данных по запросу
'rawData'	Режим работы с "сырыми" данными

▼ Пример для dataType === 'query'

```

1  import * as React from 'react';
2  import { Button } from 'Controls/buttons';
3  import { NotificationOpener } from 'Controls/popup';
4  import { copy } from 'SabyBuffer/actions';
5
6  function Example(){
7    const onCopy = React.useCallback(()=>{
8      copy({
9        dataType: 'query',
10       objectType: 'Food',
11       bufferObject: {
12         filter: { selection: { marked: [1,2] } },
13         source: 'Food.list'
14       },
15       fileName: 'ExampleFile'
16     }).then(()=>{
17       const notification = new NotificationOpener();
18       notification.open({
19         template:
20         'Controls/popupTemplate:NotificationSimple',
21         templateOptions: {
22           backgroundStyle: 'success',
23           text: 'Скопировано в буфер обмена',
24           icon: 'icon-Admin',
25         },
26       });
27     })
28   }, []);
29   return <Button caption={'Копировать в буфер'} onClick={onCopy}/>

```

▼ Пример для dataType === 'rawData'

```
1 import * as React from 'react';
2 import { Button } from 'Controls/buttons';
3 import { NotificationOpener } from 'Controls/popup';
4 import { RecordSet } from 'Types/collection';
5 import { copy } from 'SabyBuffer/actions';
6
7 function Example(){
8     const copiedItems = React.useMemo(()=> {
9         return new RecordSet({
10             rawData: { /*Данные*/ },
11             keyProperty: 'key'
12         });
13     }, []);
14     const onCopy = React.useCallback(()=>{
15         copy({
16             dataType: 'rawData',
17             objectType: 'Food',
18             bufferObject: {
19                 items: copiedItems
20             },
21             fileName: 'ExampleFile'
22         }).then(()=>{
23             const notification = new NotificationOpener();
24             notification.open({
25                 template:
26                 'Controls/popupTemplate:NotificationSimple',
27                 templateOptions: {
28                     backgroundStyle: 'success',
29                     text: 'Скопировано в буфер обмена',
30                     icon: 'icon-Admin',
31                 },
32             });
33         }, [copiedItems]);
34     return <Button caption={'Копировать в буфер'} onClick={onCopy}/>
35 }
```

TCopyMode

Значение	Описание
'extend'	Режим копирования, при котором новый объект добавляется перед предыдущим в буфере
'replace'	Режим копирования, при котором новый объект замещает предыдущий

Команда "Вставки из буфера"

Позволяет пользователю импортировать данные из буфера обмена СБИС либо системного буфера обмена.

```
1 import { paste } from 'SabyBuffer/actions';
```

Для работы команды требуется соблюдение контракта с бизнес-логикой:

Она принимает следующий набор параметров и возвращает `Promise<Record<string, Record<string, string>> | void>`:

IPasteParams

Параметр	Тип	Описание
objectName	string	Название конечной точки, обеспечивающей доступ клиента к функциональным возможностям источника данных. Объект БЛ, реализующий метод, импортирующий данные из буфера обмена. Если не указать — <code>paste()</code> не вызовет <code>ImportFromBufferLRS</code>
fiter?	Record<string, unknown>	Фильтр. Позволяет отобразить записи, которые можно вставлять. Например, в буфер обмена скопировали несколько сущностей типа "Продукты": шоколадка, конфета, рыба, мясо. После вставки из буфера в список содержимого сладкого подарка должны попасть только шоколадка и конфета. Для отбора необходим фильтр.
file?	IResource	Json-файл, расположенный в буфере обмена Если данный параметр указан, то импорт будет осуществляться из переданного Апиксабанфайла (см. пример внедрения). Иначе будет использован файл, помещенный в буфер обмена последним
customParams?	Record<string, unknown>	Пользовательские параметры для вызова метода БЛ, импортирующего данные из буфера обмена
objectTypes?	TObjectType[] (см. описание ниже)	Типы данных, доступных для вставки из буфера обмена.
clipboardData?	DataTransfer	Данные системного буфера обмена из нативного события <code>onPaste</code> . Необходимая опция при работе с системным буфером обмена.

▼ Пример кода

```

1 import * as React from 'react';
2 import { Button } from 'Controls/buttons';
3 import { NotificationOpener } from 'Controls/popup';
4 import { paste } from 'SabyBuffer/actions';
5
6 function Example(){
7     const onPaste = React.useCallback(()=>{
8         paste({
9             objectType: 'Confection',
10            filter: {
11                type: 'confection'
12            }
13        }).then(()=>{

```

```

14         const notification = new NotificationOpener();
15         notification.open({
16             template:
17 'Controls/popupTemplate:NotificationSimple',
18             templateOptions: {
19                 backgroundStyle: 'success',
20                 text: 'Импортировано из буфер обмена',
21                 icon: 'icon-Admin',
22             },
23         });
24     }, []);
25     return <Button caption={'Вставить из буфера'} onClick={onPaste}/>
26 }

```

TObjectType

Значение	Описание
'ExcelData'	Команда вставки будет пытаться сформировать JSON из данных электронной таблицы Excel / LibreOffice, либо из диапазона ячеек такой таблицы в системном буфере обмена

Обработка нажатия клавиш Ctrl+C и Ctrl+V

```

1 import CopyPasteContainer from 'SabyBuffer/CopyPasteContainer';

```

Компонент-обертка принимает опции:

Параметр	Тип	Описание
onCopy	() => void	Колбэк-функция, вызываемая при нажатии клавиш Ctrl+C
onPaste	(target?: EventTarget null, clipboardData?: DataTransfer) => void	Колбэк-функция, вызываемая при нажатии клавиш Ctrl+V

▼ Пример кода

```

1 import CopyPasteContainer from 'SabyBuffer/CopyPasteContainer';
2 import { copy, paste } from 'SabyBuffer/actions';
3
4 function Example(){
5     const onCopy = React.useCallback(()=>{
6         copy({
7             dataType: 'query',
8             objectType: 'exampleType',
9             bufferObject: {
10                 filter: { selection: { marked: [1,2] } },
11                 source: 'Example.source'
12             },
13             fileName: 'ExampleFile'
14         })
15     }, []);
16     const onPaste = React.useCallback(
17         (

```



```

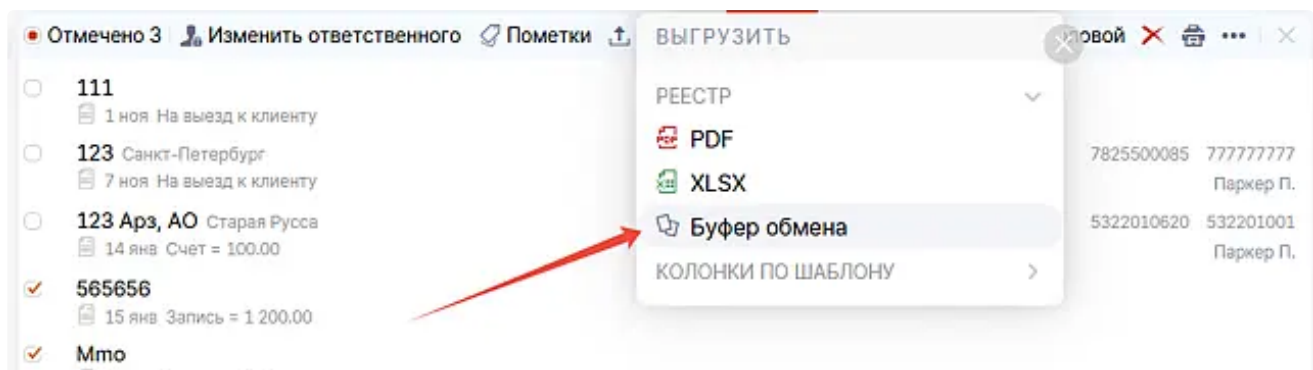
18         target?: EventTarget | null,
19         clipboardData?: DataTransfer
20     ) => {
21         paste({
22             objectName: 'exampleType2',
23             filter: {
24                 example: true
25             }
26         });
27     },
28     []
29 );
30 return (
31     <CopyPasteContainer onCopy={onCopy} onPaste={onPaste}>
32         { /* Контент */ }
33     </CopyPasteContainer>
34 )
35 }
36

```

Пример работы в режиме "параметры запроса"

Реестр [Клиенты](#)

Сценарий использования: пользователь открывает реестр клиентов, выбирает несколько записей, нажимает кнопку "копировать в буфер обмена":



По клику на эту кнопку вызывается обработчик, который использует команду "копировать в буфер" со следующими параметрами:

```

1  import { SbisService } from 'Types/source';
2  import { copy } from 'SabyBuffer/actions';
3
4  interface ICopyToBufferParams {
5      filter: object;
6      checkSelectionText?: boolean;
7  }
8
9  static async copyToBuffer(params: ICopyToBufferParams) {
10      try {
11          if (!checkBufferOperationsAccess([
12              'read',
13              'write'
14          ])) {
15              return;
16          }
17          if (params.checkSelectionText) {
18              const selectionText = document.getSelection().toString();

```

```

19         if (selectionText && selectionText.length) {
20             return;
21         }
22     }
23     await copy({
24         dataType: 'query',
25         objectType: 'CRMClients'
26         bufferObject: {
27             source: 'CRMClients.ListClients',
28             filter: params.filter
29         },
30         fileName: 'Клиенты/CRM'
31     });
32     const {Notification} = await Utils.getPopup();
33     Notification.openPopup({
34         opener: this,
35         template: 'Controls/popupTemplate:NotificationSimple',
36         autoClose: true,
37         templateOptions: {
38             backgroundStyle: 'success',
39             text: 'Выбранные клиенты скопированы в буфер обмена'
40         }
41     });
42 } catch (error) {
43     dataSource_1.error.process({ error });
44 }
45 }

```

В результате в буфере обмена появляются скопированные записи.



▼ Содержимое файла

```

1  {"d":
2  ["Food",
3  "Test.source",
4  {"selection":{"marked":["product_2","product_3"]}},
5  "s":[{"t":"Строка","n":"ObjectType"},
6  {"t":"Строка","n":"MethodName"},
7  {"t":"Запись","n":"Filter"}], "_type":"record"}

```

Затем мы можем использовать результат копирования в других реестрах.

Например, зайдя в реестр [Списки клиентов](#), мы можем вставить скопированных ранее клиентов.

The screenshot shows a CRM application interface. On the left, there is a sidebar with navigation options: "Организация", "ИП", "Физическое лицо", "Загрузить", "Папка", and "Тег". Below these are various filters and tags, including "пачки звонков 1878", "лала 8", "не бывало никог 5", "ЗАВ клиенты 4", and "крутой 3". The main area displays a list of clients with columns for name, contact information, and status. A context menu is open over the list, showing options: "Из файла", "Из буфера обмена", "Из 1С", "Из Store House", and "Из iiko". Red arrows point from the menu items to the client list, indicating the import action. The client list includes entries like "Колобкова-Крупчатникова Вильгель...", "Кригеров Анна Олеговна", "Курбатова Светла...", "Мечел-Кокс, ООО", "ММК-Метиз, ОАО", "Невада", "Новатэк- Челябинск, ООО", "Прилучный Игорь Арс...", "Северсталь Дистрибуция, ...", and "Северсталь-Метиз, ОАО".

Для отображения содержимого буфера нужно использовать список `DOCVIEW3/sbisBufferList:List`.

Клик на элемент списка вызывает команду "Вставить из буфера", передавая ей в качестве параметра файл, из которого нужно импортировать данные.



Пример

```

1 import type { IItemTemplateProps } from 'Controls/baseList';
2 import { ItemTemplate } from 'Controls/dropdown';
3 import { paste } from 'SabyBuffer/actions';
4
5 export function UploadTemplate(
6   props: IItemTemplateProps,
7   ref: React.ForwardedRef<HTMLDivElement>
8 ): JSX.Element {
9   const selectFileCallback = React.useCallback((objectType:
10 string) => {
11     return (file: object) => {
12       paste({
13         objectType,

```

```

13         file
14     })
15 }
16 }, []);
17 const templateOptions = React.useMemo(() => {
18     return {
19         selectCallback: selectFileCallback('CRMClients')
20         doNotOpenBufferWindow: true,
21         viewMode: 'List',
22         fillBufferListOnMount: true,
23     };
24 }, [item]);
25
26 return (
27     <div className="tw-w-full">
28         <ItemTemplate {...props} ref={ref} />;
29         <Async
30             templateName="DOCVIEW3/sbisBufferList:List"
31             className="controls-padding_left-s"
32             templateOptions={ templateOptions }
33         />
34     </div>
35 );
36 }

```

Клик на запись "Из буфера" вызывает команду "Вставить из буфера", которая импортирует последний файл, скопированный в буфер обмена.



▼ Пример

```

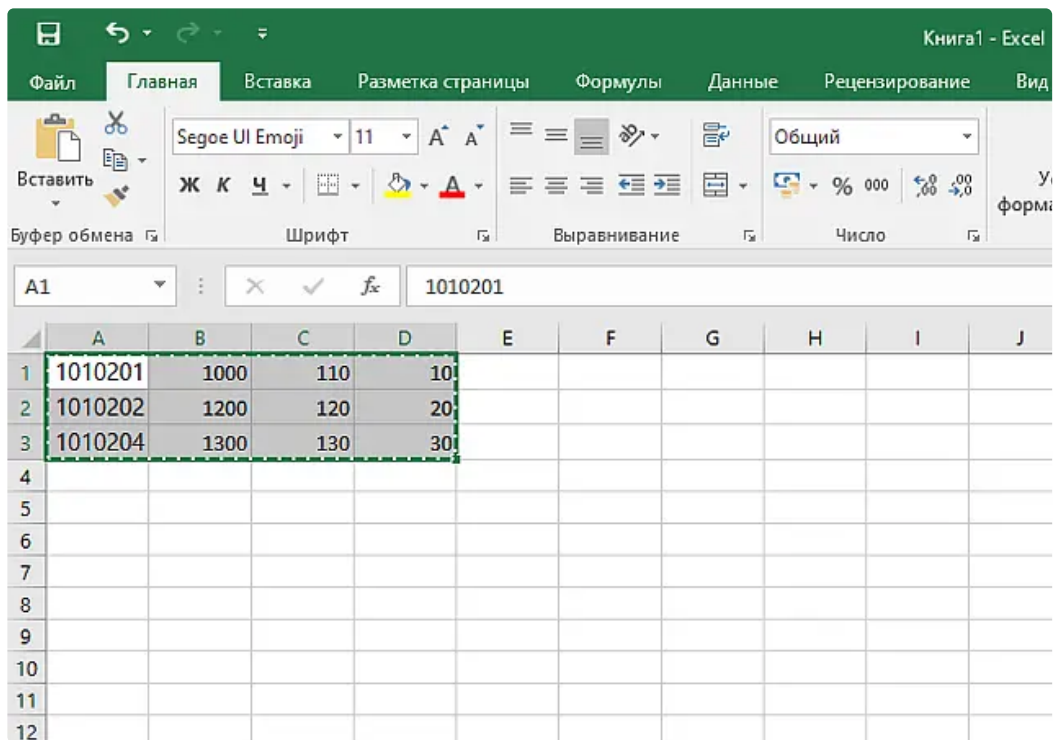
1 import { IoC } from 'Env/Env';
2 import { paste } from 'SabyBuffer/actions';
3
4 static async importFromBuffer(): void {
5     try {
6         await paste({
7             objectType: 'CRMClientImporter'
8         });
9     } catch (e) {
10         IoC.resolve('ILogger').error('Не удалось открыть
окно буфера обмена');
11     }
12 }
13

```

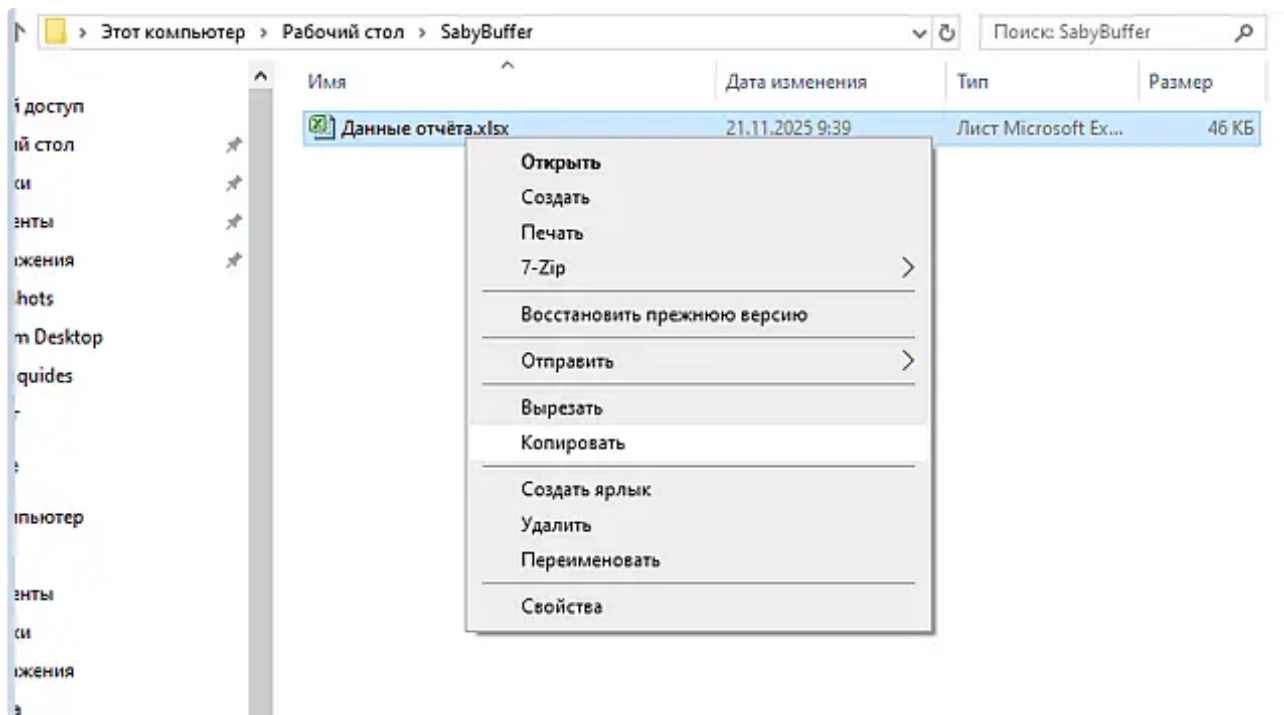
Пример работы с данными из системного буфера обмена

Реестр "[Отчётность](#)"

Сценарий использования: пользователь открывает электронную таблицу Excel / LibreOffice, выбирает диапазон ячеек и копирует в системный буфер обмена (Ctrl+C).



Либо копирует файл электронной таблицы целиком.



Затем в реестре отчётности создаёт отчёт и вставляет (CTRL+V) скопированные данные:

Найти

2025

26 ноя

ср

Расчет доли дохода

Уведомление о пори

Уведомление о пори

НД по НДС

НД по НДС

НД по НДС

НД по НДС

корректировка № 1 от

25 ноя

вт

НД по НДС

Бухгалтерская (фин

корректировка № 1 -

Первичный отчет от 1

Расчет о суммах до

Уведомление о пори

Уведомление о пори

Уведомление о пори

Уведомление о пори

Уведомление о пори

Уведомление о пори

Отчет создан

ОБЩЕСТВО С ОГРАНИЧЕННОЙ ОТВЕТСТВЕННОСТЬЮ "АТЛАНТИС"

Сохранить

Проверить

К отправки

Налоговая декларация по налогу на добавленную стоимость за III кв'25

Раздел 7

Операции, не подлежащие или освобождаемые от налогообложения

Заполняют организации и ИП, совершающие операции, необлагаемые НДС

Авансы по операциям с товарами длительного производственного цикла

0

+ Добавить

Поиск

Всего 3

Код операции (1)	Продано товаров, без НДС (2)	Приобретено необлагаемых товаров (3)	Не подлежит вычету (4)
1010201 - Предоставление в аре...	1 000	110	10
1010202 - Реализация услуг по а...	1 200	120	20
1010204 - Реализация медицинс...	1 300	130	30

Разделы с 8 по Первичные документы отсутствуют

+ Создать раздел

Скопированные из электронной таблицы данные возвращаются в виде JSON в Promise команды вставки из буфера (`SabyBuffer/actions:paste`). Поддерживаются электронные таблицы в форматах ods, xls, xlsx, csv. Если в системном буфере обмена нет данных, то в Promise вернется void.

```
{
  "1": {
    "A": "1010201",
    "B": "1000",
    "C": "110",
    "D": "10"
  },
  "2": {
    "A": "1010202",
    "B": "1200",
    "C": "120",
    "D": "20"
  },
  "3": {
    "A": "1010203",
    "B": "1300",
    "C": "130",
    "D": "30"
  }
}
```

В обработчике Promise прикладной код раскладывает полученный JSON по ячейкам таблицы и обновляет состояние формы. В результате скопированные данные появляются в отчете.

▼ Пример кода

```
1 import CopyPasteContainer from 'SabyBuffer/CopyPasteContainer';
2 import { copy, paste } from 'SabyBuffer/actions';
3
4 function Example(){
5   const onCopy = React.useCallback(()=>{
6     copy({
7       dataType: 'query',
8       objectType: 'exampleType',
9       bufferObject: {
10         filter: { selection: { marked: [1,2] } },
11         source: 'Example.source'
12       },
13       fileName: 'ExampleFile'
14     })
15   })
16 }
```

```

15     }, []);
16     const onPaste = React.useCallback(
17         (
18             target?: EventTarget | null,
19             clipboardData?: DataTransfer
20         ) => {
21             paste({
22                 objectName: 'exampleType2',
23                 filter: {
24                     example: true
25                 },
26                 objectTypes: ['ExcelData'],
27                 clipboardData, // важно: пробрасываем DataTransfer с
html/plain
28             }).then((sysBufferData: Record<string, Record<string,
string>>) => {
29                 if (!sysBufferData) {
30                     return; // вернулся Promise<void>, т.к. нет данных
31                 }
32                 // Прикладная логика
33                 // обновление состояния формы
34             });
35         },
36         []
37     );
38     return (
39         <CopyPasteContainer onCopy={onCopy} onPaste={onPaste}>
40             { /* Контент */ }
41         </CopyPasteContainer>
42     )
43 }
44

```